Project Title: Security Automation Using Bash And Cron Jobs

Student Name: Naveen Karthick S

Register Number / ID: 99220041833

Training Name: ICL - Cybersecurity Bug Bounties

College Name: Kalasalingam Academy Of Research And Education

SECURITY AUTOMATION USING BASH AND CRON JOBS

1. Abstract / Project Summary

This project focuses on automating the detection and prevention of brute-force attacks on SSH services using Bash scripting and Cron jobs in a Linux environment.

A simulated attack was performed from Kali Linux targeting an Ubuntu machine. The system continuously monitors SSH login attempts from log files and automatically blocks suspicious IP addresses using firewall rules (iptables).

The automation ensures real-time protection without manual intervention, making it suitable for real-world security monitoring systems.

2. Introduction

In modern systems, servers are constantly exposed to attacks such as brute-force login attempts. One common target is SSH (Secure Shell), which allows remote access to systems.

Tools like Hydra are used by attackers to try multiple password combinations rapidly.

To defend against such attacks:

- Monitoring logs manually is inefficient
- Automated systems are required

This project uses:

- Bash scripting → for automation logic
- Cron jobs → for continuous execution
- Firewall rules → for blocking attackers

3. Objectives

- Detect multiple failed SSH login attempts
- Identify attacker IP addresses
- Automatically block malicious IPs
- Maintain logs of blocked attackers
- Implement whitelist for safe IPs
- Automate monitoring using cron

4. Tools & Technologies Used

Tool	Purpose
Kali Linux	Simulate attacker
Ubuntu	Target machine
Bash Scripting	Automation logic
Cron Jobs	Scheduled execution
iptables	Firewall to block IPs
SSH	Remote login service
Hydra	Brute-force attack tool

5. Methodology / Steps Performed

Step 1: Setup Internal Network

- Created two virtual machines:
  - Kali Linux (attacker)
  - Ubuntu (target)
- Connected both using **Internal Network**

Purpose:

- Simulate real-world attacker–victim environment

Step 2: Assign IP Address

What we are doing

We are giving IP addresses to both systems so they can talk to each other.

- **Kali (Attacker)** → 192.168.10.10
- **Ubuntu (Target)** → 192.168.10.20

Both must be in the same network.

Kali Linux (Attacker)

Command

```
sudo ip addr add 192.168.10.10/24 dev eth0
sudo ip link set eth0 up
```

Simple Meaning

- **First command** → gives IP address to Kali
- **Second command** → turns ON network

Ubuntu (Target)

Command

sudo ip addr add 192.168.10.20/24 dev enp0s3
sudo ip link set enp0s3 up

Simple Meaning

- **First command** → gives IP address to Ubuntu
- **Second command** → turns ON network

Check IP (Both Systems)

Command - ip a

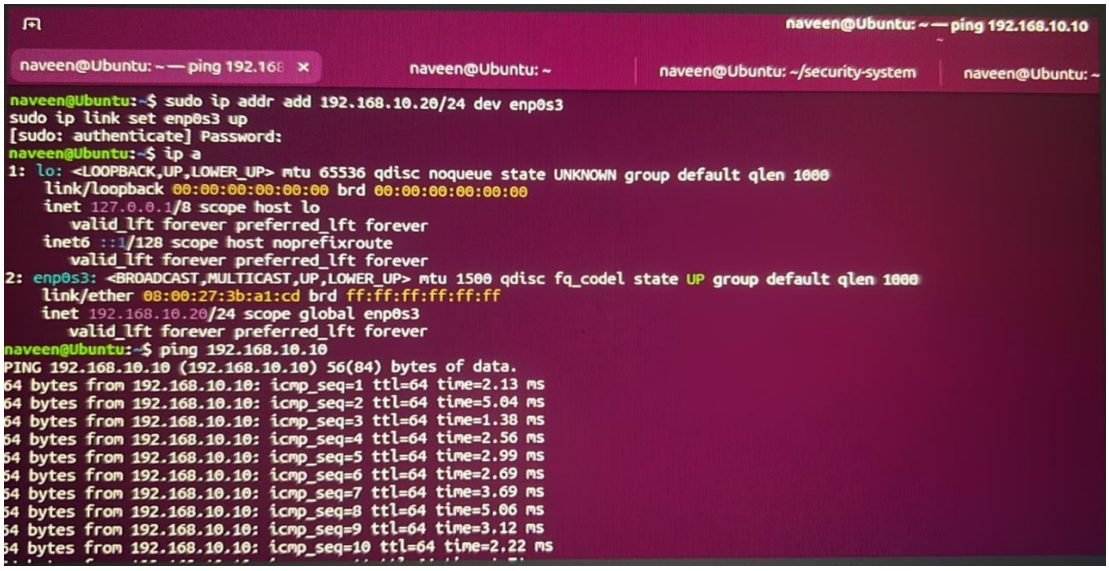
You should see

- **Kali** → 192.168.10.10
- **Ubuntu** → 192.168.10.20

Test Connection From Kali

ping 192.168.10.20

Output :

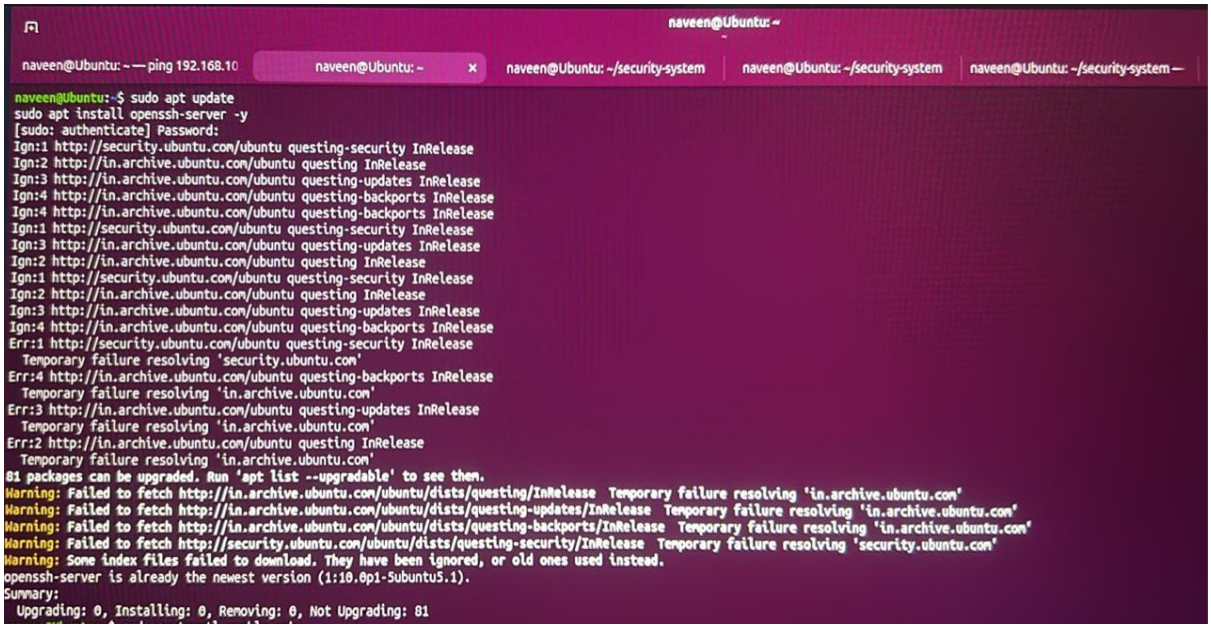


Step 3: Install & Start SSH Service (Ubuntu)

Install SSH Command:

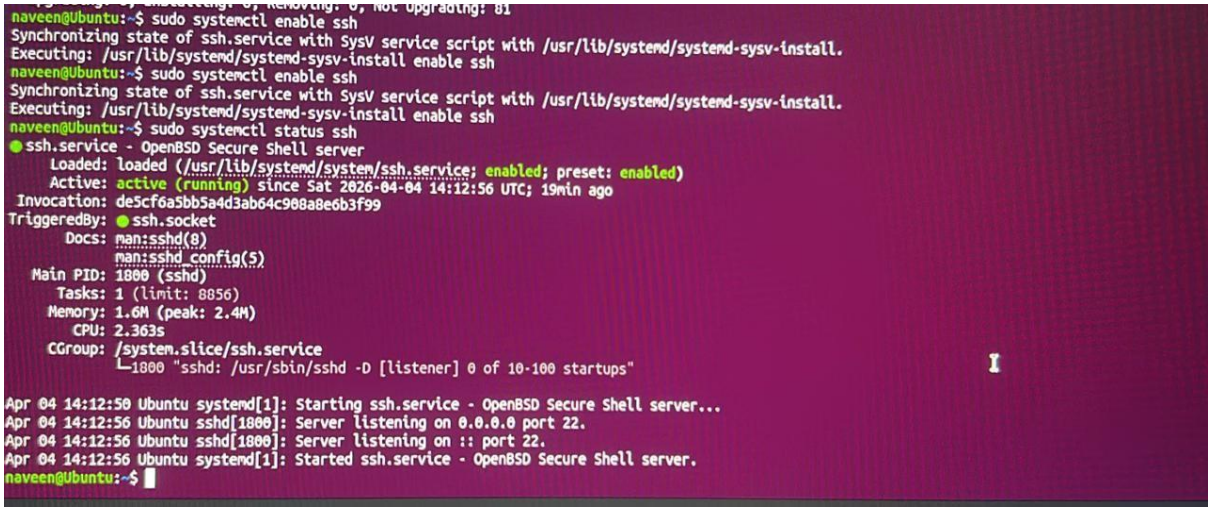
sudo apt update

sudo apt install openssh-server



Start SSH Command:

sudo systemctl start ssh
sudo systemctl status ssh



Purpose:

- Enable remote login service (target for attack)

Step 4: Perform Attack from Kali

Command:

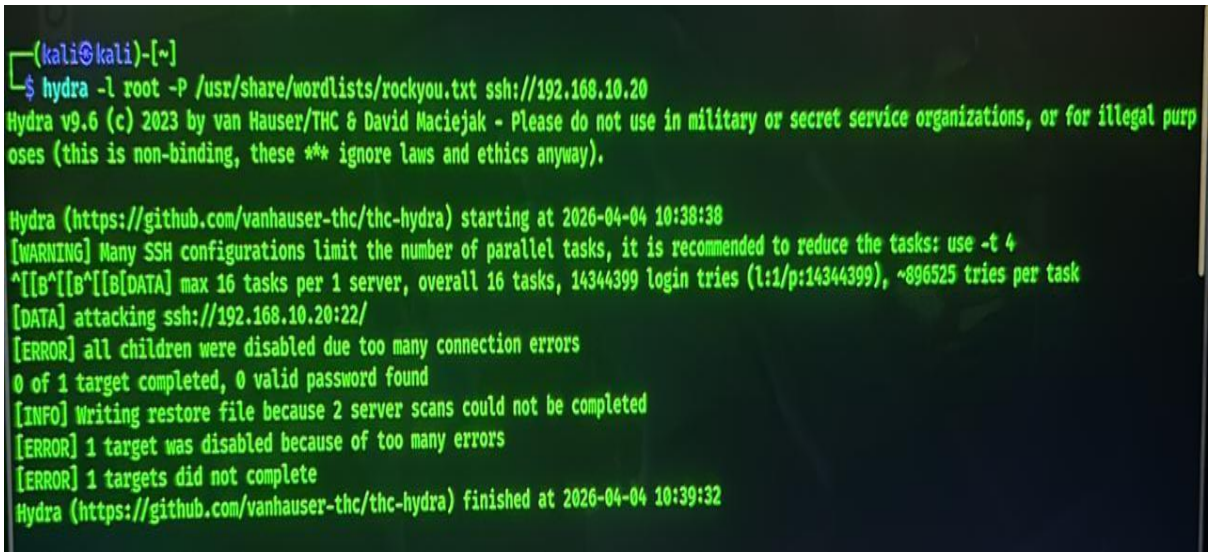
hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.10.20

Purpose:

- Simulate brute-force attack

Output:

- Multiple failed login attempts recorded in logs



```
┌──(kali㉿kali)-[~]
└─$ hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.10.20
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purp
oses (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-04 10:38:38
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
^[[B^[[B^[[B[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ssh://192.168.10.20:22/
[ERROR] all children were disabled due too many connection errors
0 of 1 target completed, 0 valid password found
[INFO] Writing restore file because 2 server scans could not be completed
[ERROR] 1 target was disabled because of too many errors
[ERROR] 1 targets did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-04 10:39:32
```

Step 5: Create Project Directory (Ubuntu)

Command:

mkdir ~/security-system
cd ~/security-system

Purpose:

- Store all scripts and logs in one place

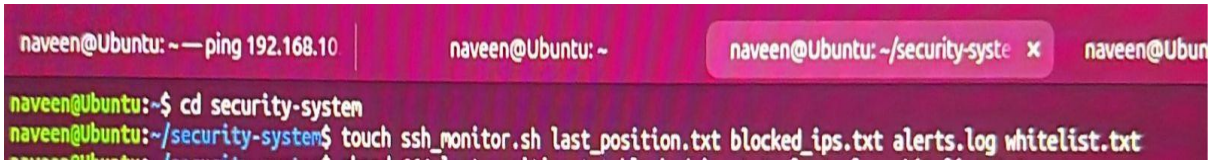
Step 6: Create Required Files

Command:

touch ssh_monitor.sh blocked_ips.txt alerts.log whitelist.txt last_position.txt

Purpose:

- **Store:**
    - Blocked IPs
    - Alerts
    - Whitelist
    - Last log position



Step 7: Check Log File (Ubuntu)

Command:

cat /var/log/auth.log

Example Output:

Failed password for root from 192.168.1.5

Purpose:

- Shows failed login attempts
- Confirms attack is happening
- Provides attacker IP address

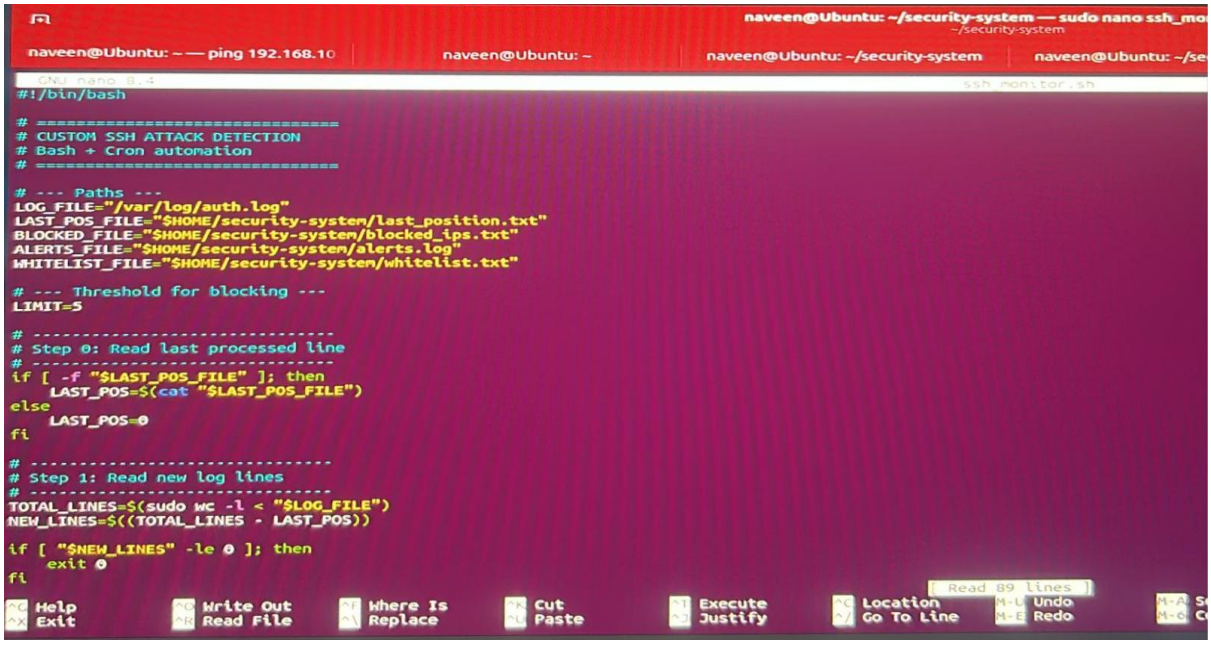
Step 8: Create Monitoring Script

Command:

nano ssh_monitor.sh

Purpose:

- Create a Bash script that:
    - Reads log file
    - Detects failed attempts
    - Counts attempts per IP
    - Blocks attacker



Monitoring.sh script :

```
#!/bin/bash
# ==========
# CUSTOM SSH ATTACK DETECTION
# Bash + Cron automation
# ==========
# --- Paths ---
LOG_FILE="/var/log/auth.log"
LAST_POS_FILE="$HOME/security-system/last_position.txt"
BLOCKED_FILE="$HOME/security-system/blocked_ips.txt"
ALERTS_FILE="$HOME/security-system/alerts.log"
WHITELIST_FILE="$HOME/security-system/whitelist.txt"
# --- Threshold for blocking ---
LIMIT=5
# ----------
# Step 0: Read last processed line
# ----------
if [ -f "$LAST_POS_FILE" ]; then
   LAST_POS=$(cat "$LAST_POS_FILE")
else
   LAST_POS=0
fi
```

```
# -------------------------------
# Step 1: Read new log lines
# -------------------------------
TOTAL_LINES=$(sudo wc -l < "$LOG_FILE")
NEW_LINES=$((TOTAL_LINES - LAST_POS))
if [ "$NEW_LINES" -le 0 ]; then
    exit 0
fi
# Read only new lines
TAIL_OUTPUT=$(sudo tail -n "$NEW_LINES" "$LOG_FILE" | grep -a "Failed password")
# Update last position
echo "$TOTAL_LINES" > "$LAST_POS_FILE"
# -------------------------------
# Step 2: Extract attacker IPs
# -------------------------------
ATTACKERS=$(echo "$TAIL_OUTPUT" | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+')
declare -A IP_COUNT
for IP in $ATTACKERS; do
    ((IP_COUNT[$IP]++))
done
# -------------------------------
# Step 3: Read blocked IPs and whitelist
# -------------------------------
if [ -f "$BLOCKED_FILE" ]; then
    BLOCKED=$(cat "$BLOCKED_FILE")
else
    BLOCKED=""
fi
if [ -f "$WHITELIST_FILE" ]; then
    WHITELIST=$(cat "$WHITELIST_FILE")
else
    WHITELIST=""
Fi
```

```
# -------------------------------
# Step 4: Check threshold and block
# -------------------------------
for IP in "${!IP_COUNT[@]}"; do
    # Skip whitelist
    if echo "$WHITELIST" | grep -q "$IP"; then
        continue
    fi
    # Skip already blocked
    if echo "$BLOCKED" | grep -q "$IP"; then
        continue
    fi
    # Block if attempts exceed threshold
    if [ "${IP_COUNT[$IP]}" -ge "$LIMIT" ]; then
        sudo iptables -A INPUT -s $IP -j DROP
        echo "$IP" >> "$BLOCKED_FILE"
        echo "$(date) - Blocked $IP (${IP_COUNT[$IP]} attempts)" >> "$ALERTS_FILE"
        echo "Blocked $IP (${IP_COUNT[$IP]} attempts)"
    fi
done
```

Step 9: Give Permission To Script

Command:

chmod +x ssh_monitor.sh

Purpose:

- Makes the script executable

Step 10: Execute Script and Verify Firewall Blocking

Command:

./ssh_monitor.sh

sudo iptables -L -n

cat ~/security-system/blocked_ips.txt

cat ~/security-system/alerts.log

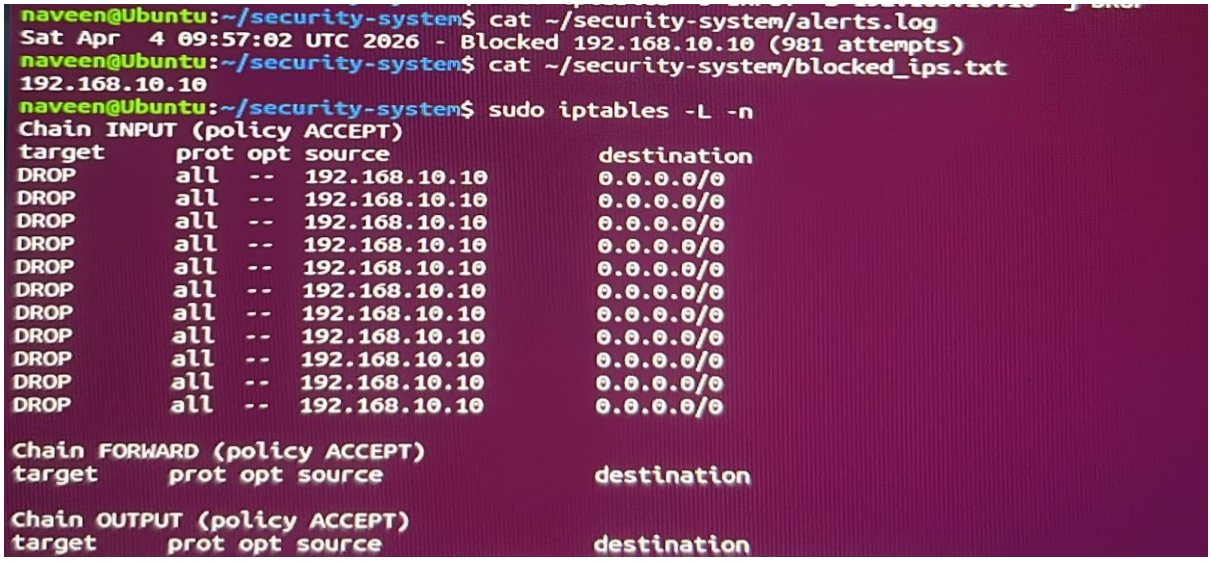
Purpose:

- To test whether the script detects brute-force SSH attacks
- To automatically block the attacker IP when the threshold is exceeded
- To verify that the firewall rule is successfully applied

Expected Output:

Attack from 192.168.10.10 (981 attempts)
Blocked 192.168.10.10

Chain INPUT (policy ACCEPT)
target prot opt source destination
DROP all -- 192.168.10.10 0.0.0.0/0



```
naveen@Ubuntu:~/security-system$ cat ~/security-system/alerts.log
Sat Apr  4 09:57:02 UTC 2026 - Blocked 192.168.10.10 (981 attempts)
naveen@Ubuntu:~/security-system$ cat ~/security-system/blocked_ips.txt
192.168.10.10
naveen@Ubuntu:~/security-system$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

Explanation:

- The script reads system logs and detects repeated failed login attempts
- Once the number of attempts exceeds the defined limit, it blocks the attacker IP
- The iptables output confirms that a DROP rule has been added
- This ensures that all traffic from the attacker IP is denied, preventing further access

Step 11: Automate using Cron

Command:

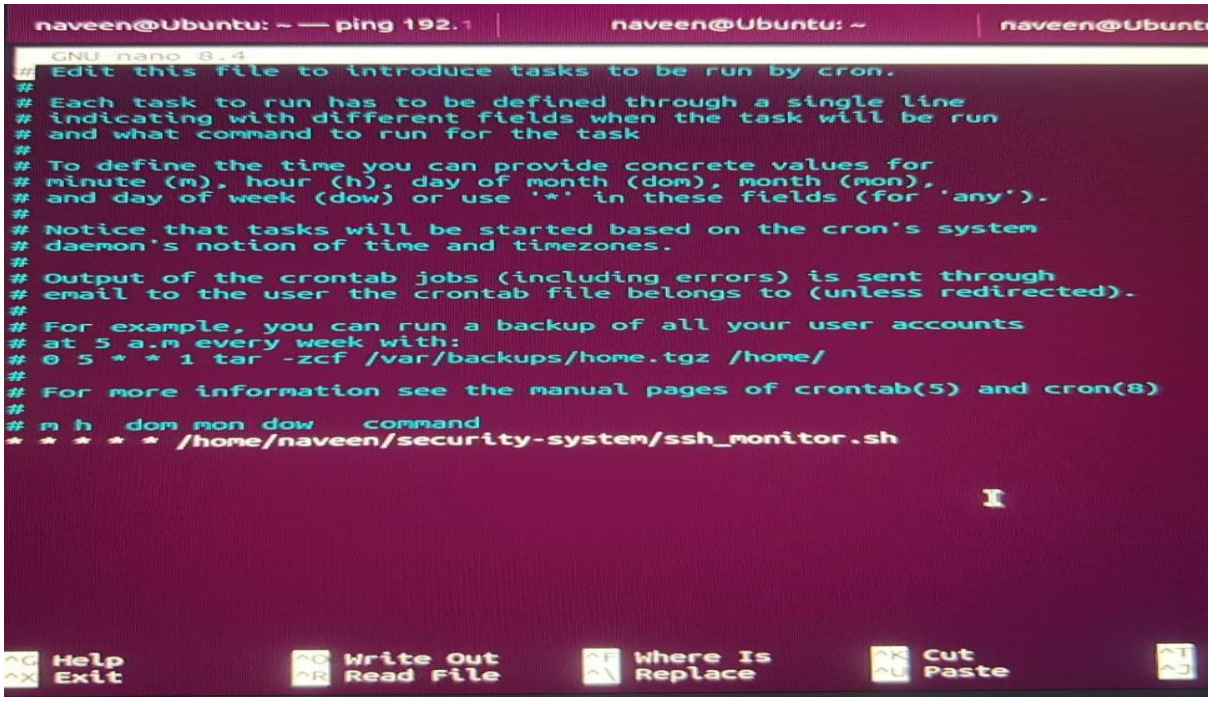
crontab -e

Add:

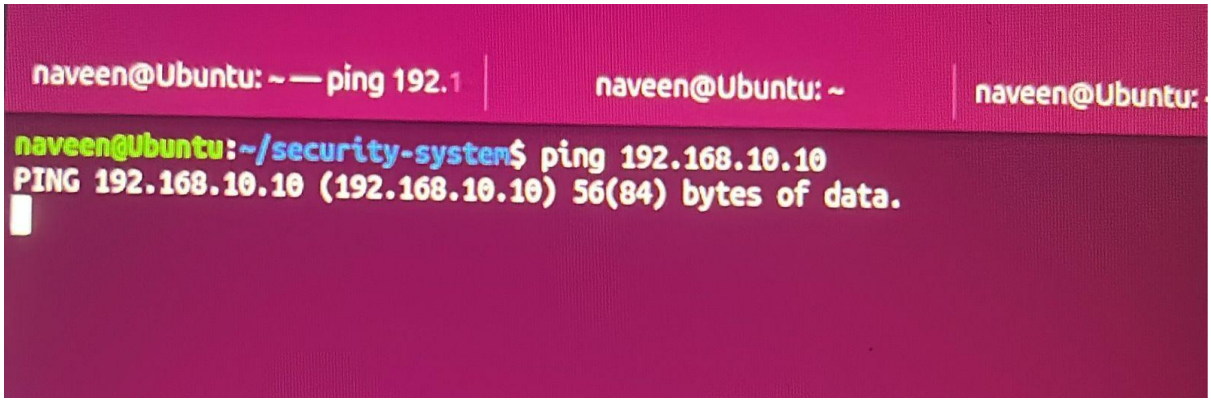
* * * * * /home/naveen/security-system/ssh_monitor.sh

Purpose:

- Run script every 1 minute



After blocking there will be no ping :



Step 12: Implement Whitelist for Safe IPs

Command:

nano ~/security-system/whitelist.txt

Add trusted IPs:

192.168.10.10
192.168.10.20

Purpose:

- To define trusted IP addresses that should **never be blocked**
- Prevents accidental blocking of legitimate users (like your own Kali or admin system)

How it Works:

- The script reads whitelist.txt before blocking any IP
- If an IP is found in the whitelist, it is **ignored** even if it exceeds the attack threshold

Verification:

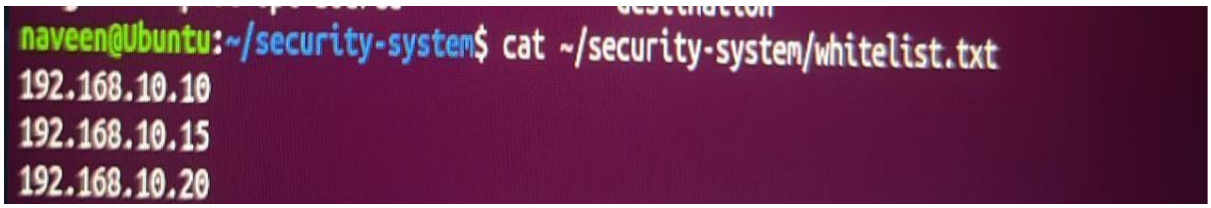
cat ~/security-system/whitelist.txt

Expected Output:

192.168.10.10
192.168.10.20

Result:

- Whitelisted IPs will **bypass blocking rules**
- Only unknown or malicious IPs will be blocked



6. Findings / Analysis

- Multiple failed login attempts were detected from attacker IP
- Script successfully identified repeated attempts
- IP was automatically blocked after threshold (e.g., 5 attempts)

- Logs confirmed detection and blocking

Example log:

Blocked 192.168.10.10 (981 attempts)

7. Results

- Attack simulation was successful
- Detection system worked correctly
- Firewall rules blocked attacker
- Automation via cron ensured continuous monitoring

Final verification:

sudo iptables -L -n

Output:

DROP all -- 192.168.10.10

```
naveen@Ubuntu:~/security-system$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0
DROP       all  --  192.168.10.10        0.0.0.0/0

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

8. Challenges Faced

- SSH service not running initially
- IP connectivity issues
- Firewall rules blocking own system
- Script not detecting logs properly
- Cron not executing due to permission issues

9. Conclusion

This project demonstrates a practical implementation of automated security monitoring using Bash scripting and Cron jobs.

It successfully:

- Detects brute-force attacks
- Blocks attackers automatically
- Maintains logs
- Runs without manual intervention

This approach is scalable and can be extended for enterprise-level security systems.

10. References

- Ubuntu Documentation – Security Logs:

  https://help.ubuntu.com/community/Security/Logs
- iptables Manual: https://linux.die.net/man/8/iptables
- OpenSSH Manual: https://www.openssh.com/manual.html
- Advanced Bash Scripting Guide: https://tldp.org/LDP/abs/html/
- Cron Manual Page: https://man7.org/linux/man-pages/man5/crontab.5.html